CSE145 - Introduction to Computing

Capstone Milestone M1: Gates and Combinational Logic - Report

Name: Tushar Kumar ERP ID: 37069 Tool: Logisim (V5.0.0) File: m1_37069.circ

Git Repo: https://github.com/unluckyprog/CSE145-toy-machine.git

Part 1 - Boolean algebra on paper

1.1 De Morgan's laws (proof by truth table)

x	y	xy	(xy)'	x'	y'	x'+y'
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

x	y	x+y	(x+y)'	x'	y'	x'y'
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

The (xy)' column and the x'+y' column are the same in all 4 rows, and the (x+y)' column is the same as x'y'. So both laws are true.

1.2 Counting

(a) With N inputs there are 2^{2^N} functions.

For N = 2, functions are 2^4 = 16; N = 3, functions are 2^8 = 256; N = 4, functions are 2^{16} = 65,536.

Some functions for N=2 are NOT, AND, OR, NOR, XOR, NAND.

(b) A complete n-bit adder truth table needs 2^{2n} rows:

Adder	Input bits	Rows
4-bit	8	2^8 = 256
8-bit	16	2^16 = 65,536
16-bit	32	2^32 = 4,294,967,296

1.3 My personal function F

My ERP = 37069, so the last three digits are 069 and D = 69. B = 69 mod 256 = 69, and 69 = 01000101 in binary. The output column is 0,1,0,0,0,1,0,1.

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Rows with F = 1 are 001, 101 and 111, so **F = x'y'z + xy'z + xyz**.

It has **3 product terms**.

1.4 Half adder and full adder tables

Half Adder

x	y	S	C	x XOR y	xy
0	0	0	0	0	0
0	1	1	0	1	0
1	0	1	0	1	0
1	1	0	1	0	1

Full Adder

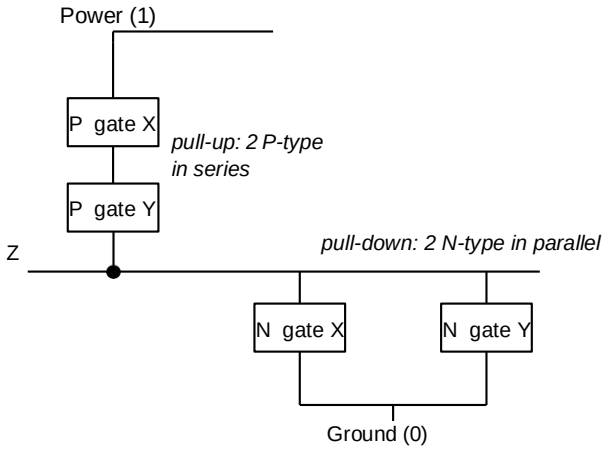
x	y	cin	S	cout	ODD	MAJ
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	1	0	1	0
0	1	1	0	1	0	1
1	0	0	1	0	1	0
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	1	1	1	1

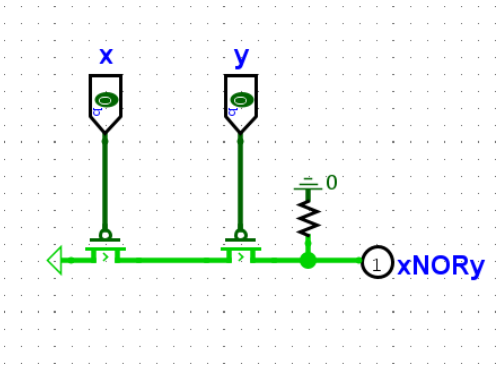
In the half adder, S is the same as the XOR column and C is the same as the AND column.
In the full adder, S matches ODD and cout matches MAJ in all 8 rows.

Sum of products:
Half adder: S = x'y + xy' and C = xy
Full adder: S = x'y'c + x'yc' + xy'c' + xyc and cout = x'yc + xy'c + xyc' + xyc

Part 2 - The Ummul gate and gates built from it

2a. NOR_T from transistors

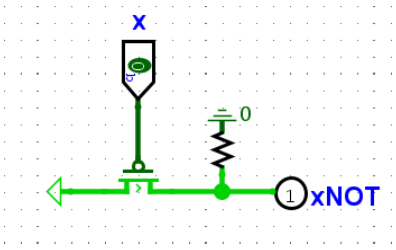




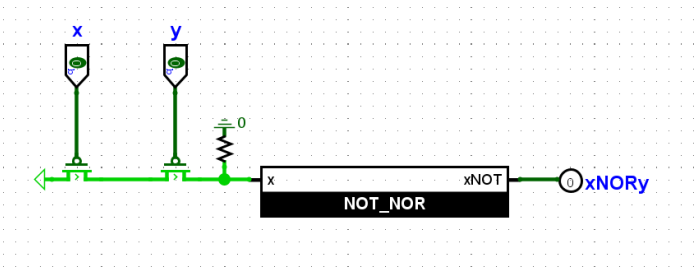
Logisim screenshot

The two P-type transistors in series connect Power to Z only when X = 0 and Y = 0, and the two N-type transistors in parallel connect Z to Ground when X = 1 or Y = 1. These two conditions are exact opposites, so for every input row exactly one network conducts and Z is never floating (blue) or shorted (red).

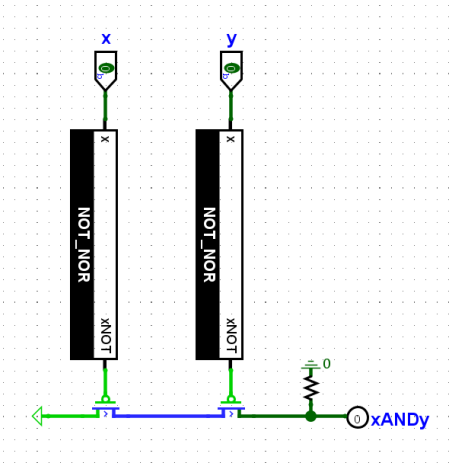
2b - 2d. NOT_NOR, OR_NOR, AND_NOR



NOT_NOR (X to the NOR input, output xNOT)



OR_NOR (NOR followed by NOT_NOR)



AND_NOR (NOT both inputs, then NOR)

Observed truth tables:

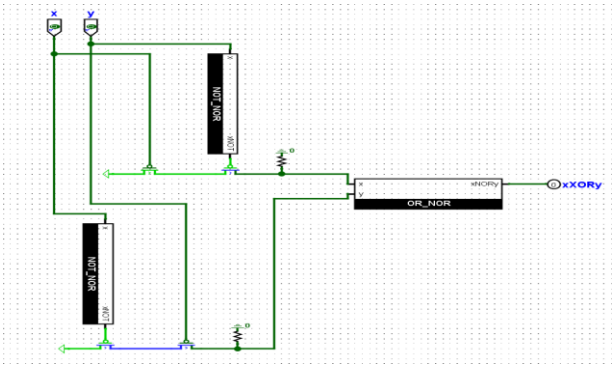
X	Y	NOR_T
0	0	1
0	1	0
1	0	0
1	1	0

X	NOT_NOR
0	1
1	0

X	Y	OR_NOR
0	0	0
0	1	1
1	0	1
1	1	1

X	Y	AND_NOR
0	0	0
0	1	0
1	0	0
1	1	1

2e. XOR_SOP



The two product terms are x'y and xy'. I noticed that x'y is the same as NOR(x, y'), so each term only needs one NOT_NOR and one NOR gate, and then one OR_NOR joins them. Observed truth table:

X	Y	XOR_SOP
0	0	0
0	1	1
1	0	1
1	1	0

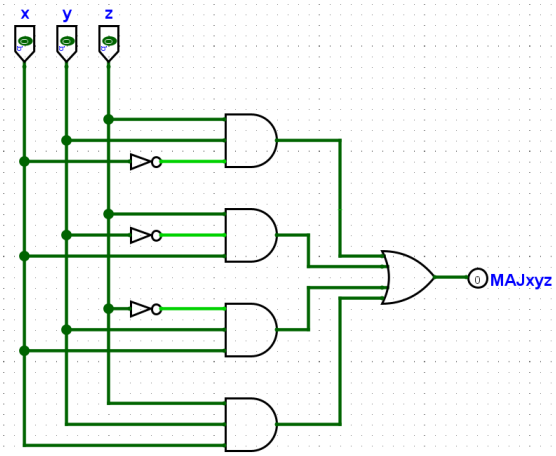
XOR_SOP: x'y + xy' (xXORy)

Gate counts after expanding everything down to NOR_T (4 transistors each):

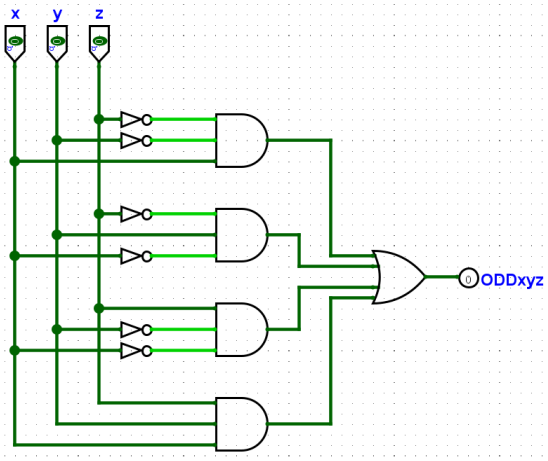
Circuit	NOR_T gates	Transistors
NOT_NOR	1	4
OR_NOR	2 (NOR + NOT)	8
AND_NOR	3 (two NOTs + NOR)	12
XOR_SOP	6 (2 NOT + 2 NOR + OR_NOR of 2)	24

Part 3 - Sum-of-products design

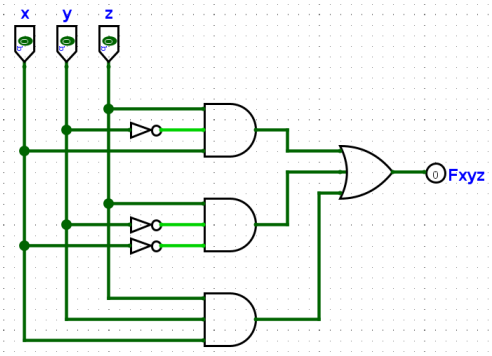
I followed the 4-step recipe: write the truth table, mark the rows with output 1, make one generalised AND for each such row, and OR them. I used Logisim's built-in AND, OR and NOT gates here.



MAJ3: x'yz + xy'z + xyz' + xyz



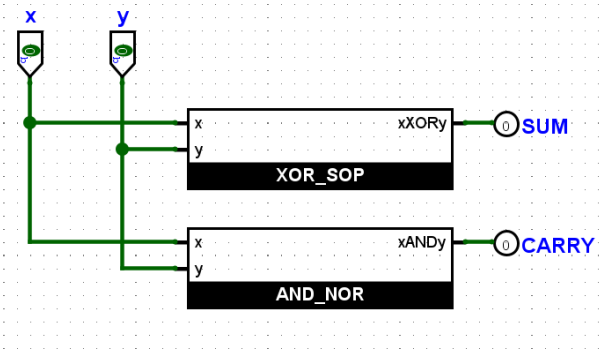
ODD3: x'y'z + x'yz' + xy'z' + xyz



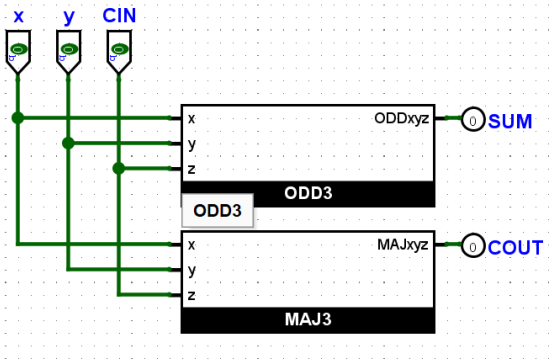
PERSONAL_F: x'y'z + xy'z + xyz

x	y	z	MAJ	ODD	F
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	0	1	0
1	0	1	1	0	1
1	1	0	1	0	0
1	1	1	1	1	1

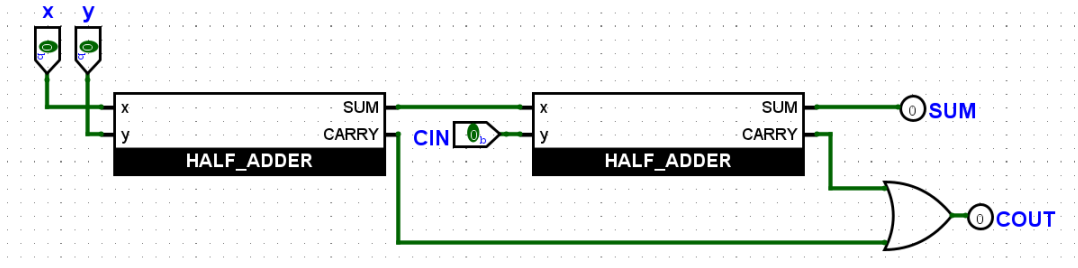
Part 4 - Half adder and full adder



HALF_ADDER: S from XOR_SOP, C from AND_NOR



FULL_ADDER_A: ODD3 for SUM, MAJ3 for COUT



FULL_ADDER_B: two HALF_ADDERs and an OR gate

4d. TESTBENCH. The three input pins X, Y, CIN drive both full adders. Both S and COUT outputs are shown on pins. MISMATCH = (S_A XOR S_B) OR (COUT_A XOR COUT_B), using built-in XOR and OR, which is allowed in the TESTBENCH. A built-in NOR gate sits next to NOR_T with the same X and Y inputs as the reference.

X	Y	CIN	S (A)	COUT (A)	S (B)	COUT (B)	MISMATCH	X+Y+CIN	2*COUT+S
0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0	1	1
0	1	0	1	0	1	0	0	1	1
0	1	1	0	1	0	1	0	2	2
1	0	0	1	0	1	0	0	1	1
1	0	1	0	1	0	1	0	2	2
1	1	0	0	1	0	1	0	2	2
1	1	1	1	1	1	1	0	3	3

MISMATCH is 0 in all 8 rows and 2*COUT + S equals X + Y + CIN every time.

NOR_T against the built-in NOR:

X	Y	NOR_T	built-in NOR	agree?
0	0	1	1	yes
0	1	0	0	yes
1	0	0	0	yes
1	1	0	0	yes

(i) Why a plain OR is enough in FULL_ADDER_B

The first half adder gives carry xy, which is 1 only when x = y = 1. But then its sum x XOR y is 0, so the second half adder is adding 0 + CIN and its carry is 0. So the two partial carries can never be 1 together. Because of that, OR gives the right answer and we never need to add the two carries.

(ii) Gate comparison

What I counted as one gate: one NOT, or one AND, or one OR. XOR_SOP is counted as 2 NOT + 2 AND + 1 OR = 5 gates with path NOT, AND, OR. The carry AND in a half adder is 1 gate.

Circuit	Gate breakdown	Total gates	Longest path (gates)
FULL_ADDER_A	MAJ3: 3 NOT + 4 AND + 1 OR = 8 ODD3: 6 NOT + 4 AND + 1 OR = 11	19	3
FULL_ADDER_B	2 half adders (5 + 1 each) = 12, plus 1 OR	13	6 (XOR 3 + XOR 3)

If everything is made only from NOR_T: XOR_SOP = 6 NOR_T = 24 transistors. FULL_ADDER_B = 2 x (XOR_SOP 6 + AND_NOR 3) + OR_NOR 2 = 20 NOR_T = 80 transistors. So universal does not mean efficient. A ready-made XOR gate needs far fewer transistors than 24, but NOR-only is simple to design since we only have one type of block.

Part 5 - Reflection

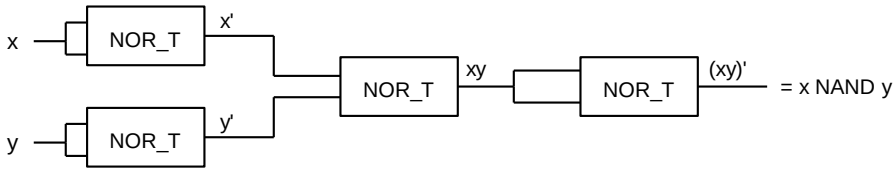
R1. A truth table lists every input combination. For each row where the output is 1 we make an AND term that is 1 only for that row, then OR all the terms, so the output is 1 on exactly those rows. The drawback is that the circuit can become huge, because the number of rows and terms doubles with every extra input.

R2. A 16-bit adder has 32 inputs, so one truth table would need 2^32 rows, which is impossible to draw. The lecture idea is to add one bit at a time with a full adder and pass the carry to the next column. Then we only need 16 small circuits.

R3. FULL_ADDER_A is better for speed: its longest path is 3 gates while B has 6. FULL_ADDER_B is better for size: 13 gates against 19. So B costs more delay, and A costs more gates.

R4. Transistor (a switch controlled by its gate input) -> NOR_T (hides the pull-up and pull-down transistors) -> NOT_NOR, OR_NOR, AND_NOR (hide how many NORs are inside) -> XOR_SOP (hides the sum-of-products wiring) -> HALF_ADDER (hides the XOR and AND, only S and C are visible) -> FULL_ADDER_B (hides the two half adders and the OR, only X, Y, CIN, S, COUT are visible).

R5. NOR is the 'mother' gate because NOT, OR and AND can all be made from it, and those three can make any Boolean function by sum of products. So everything can be built from NOR alone. NAND from four NOR_T gates: NOR(x,x) = x', NOR(y,y) = y', NOR(x',y') = xy, then NOR(xy,xy) = (xy)'. Sketch:



R6. My MAJ3 gave the wrong output for the row 110 (it showed 0 instead of 1). I used the Poke tool on each row and saw that one AND gate was not getting the y input. The wire only crossed the other wire and there was no junction dot, so it was not connected. I added the dot, tested all 8 rows again and committed it as 'Fix MAJ3 wiring'.